

Universidade Vila Velha – UVV

Projeto Acadêmico

UniHub

Documentação Técnica do Projeto

Integrantes

Matheus Pastore

Felipe Yuki

Davi Conde

Vila Velha – ES

10 de junho de 2026

Sumário

1	Descrição Geral do Produto	2
1.1	Principais Funcionalidades	2
2	Arquitetura Adotada	2
2.1	Trecho Principal da Fábrica de Aplicação	2
2.2	Estrutura de Pastas	3
3	Organização em Módulos e Rotas	3
4	Formulários e Validação	4
5	Interface e Templates	4
5.1	Arquivos Importantes	4
6	Persistência e Banco de Dados	5
6.1	Diagrama do Banco de Dados	5
6.2	Tabelas Principais	6
6.3	Regras de Integridade	6
7	Testes Automatizados	7
7.1	Cobertura Principal	7
8	Tecnologias Utilizadas	7
9	Como Rodar o Projeto	8
9.1	Instalação Recomendada no Linux/macOS	8
9.2	Instalação Recomendada no Windows PowerShell	8
9.3	Instalação Recomendada no Windows CMD	8
9.4	Instalação Manual no Linux/macOS	8
9.5	Instalação Manual no Windows PowerShell	9
9.6	Rodar o Servidor	9
9.7	Executar os Testes	9
9.8	URLs Principais	9
9.9	Contas Fictícias para Teste	10
10	Principais Desafios e Soluções	10

1 Descrição Geral do Produto

O **UniHub** é uma plataforma web para centralizar a vida acadêmica e social de estudantes universitários. O sistema reúne autenticação, perfil, dashboard, eventos, agenda pessoal, fórum acadêmico, moradias, mensagens, notificações e painel administrativo.

1.1 Principais Funcionalidades

- Cadastro, login, logout e sessão com Flask-Login.
- Perfil do usuário com edição de dados.
- Dashboard com resumo de eventos, tópicos, moradias e notificações.
- Fórum com tópicos, respostas, avisos oficiais e moderação.
- Eventos com criação por moderadores e administradores, edição, detalhes e agenda pessoal.
- Moradias com cadastro, edição, detalhes e controle de status.
- Mensagens entre usuários e notificações.
- Painel administrativo para gerenciamento de usuários, eventos e fórum.

2 Arquitetura Adotada

O projeto usa **Flask** com o padrão **Application Factory**. A função `create_app()` em `app.py` centraliza a criação da aplicação, carrega configurações, inicializa extensões, registra models, login, CSRF, blueprints, Swagger e comandos de seed.

2.1 Trecho Principal da Fábrica de Aplicação

Listing 1: Função `create_app` em `app.py`

```
def create_app(test_config=None):
    flask_app = Flask(__name__, template_folder="templates",
                      static_folder="static")
    init_config(flask_app, test_config)
    init_db(flask_app)
    register_models()
    init_login(flask_app)
    init_wtf(flask_app)
    init_views(flask_app)
```

```
init_swagger(flask_app)
register_seed_command(flask_app)
return flask_app
```

2.2 Estrutura de Pastas

- unihub/models: entidades SQLAlchemy.
- unihub/views: rotas e blueprints por domínio.
- unihub/forms: formulários Flask-WTF/WTForms.
- unihub/ext: inicialização de extensões.
- unihub/services: seed e serviços de apoio.
- unihub/utils: helpers de segurança, autenticação e respostas.
- templates: telas Jinja2 e componentes.
- tests: testes automatizados com unittest.
- migrations: migrations Alembic/Flask-Migrate.

3 Organização em Módulos e Rotas

As rotas são organizadas em **Blueprints**, garantindo separação clara de responsabilidades.

- main: landing page, dashboard, perfil, configurações e healthcheck.
- auth: cadastro, login, logout e usuário autenticado.
- usuarios: consulta e atualização de usuário.
- forum: tópicos, respostas e moderação.
- eventos e agenda: eventos e agenda pessoal.
- moradias: anúncios de moradia.
- mensagens: conversas entre usuários.
- notificacoes: notificações do usuário.
- admin: telas administrativas.

O registro dos blueprints acontece no arquivo `unihub/views/__init__.py`, mantendo a inicialização centralizada.

4 Formulários e Validação

O projeto usa **Flask-WTF/WTForms** para validação no servidor. Os formulários ficam em `unihub/forms` e são usados tanto para telas HTML quanto para contratos principais de entrada.

- `CadastroForm`: nome, email, senha, confirmação, curso, período, cidade e biografia.
- `LoginForm`: email e senha obrigatórios.
- `EventoForm`: título, descrição, categoria, data, horário e local.
- `MoradiaForm`: título, descrição, bairro, preço, vagas e características.
- `ForumTopicoForm` e `ForumRespostaForm`: validação de tópicos e respostas.

O CSRF é inicializado em `unihub/ext/wtf.py`. Em testes ele é desativado para permitir testes automatizados sem token manual.

5 Interface e Templates

A interface usa **Jinja2**, templates por módulo e componentes reutilizáveis. Existem layouts separados para páginas públicas, autenticação e área logada. A camada visual usa **Bootstrap** e CSS próprio, com telas para landing page, login, cadastro, dashboard, perfil, notificações, fórum, eventos, moradias, mensagens e administração.

5.1 Arquivos Importantes

- `templates/layouts/public.html`
- `templates/layouts/auth.html`
- `templates/layouts/app.html`
- `templates/components/`
- `templates/auth/`
- `templates/eventos/`
- `templates/forum/`
- `templates/moradias/`
- `templates/main/`

6 Persistência e Banco de Dados

O projeto usa **SQLite** em desenvolvimento/testes e **SQLAlchemy** como ORM. As migrations ficam em `migrations/`.

6.1 Diagrama do Banco de Dados

A Figura 1 apresenta o diagrama do banco de dados utilizado no projeto.

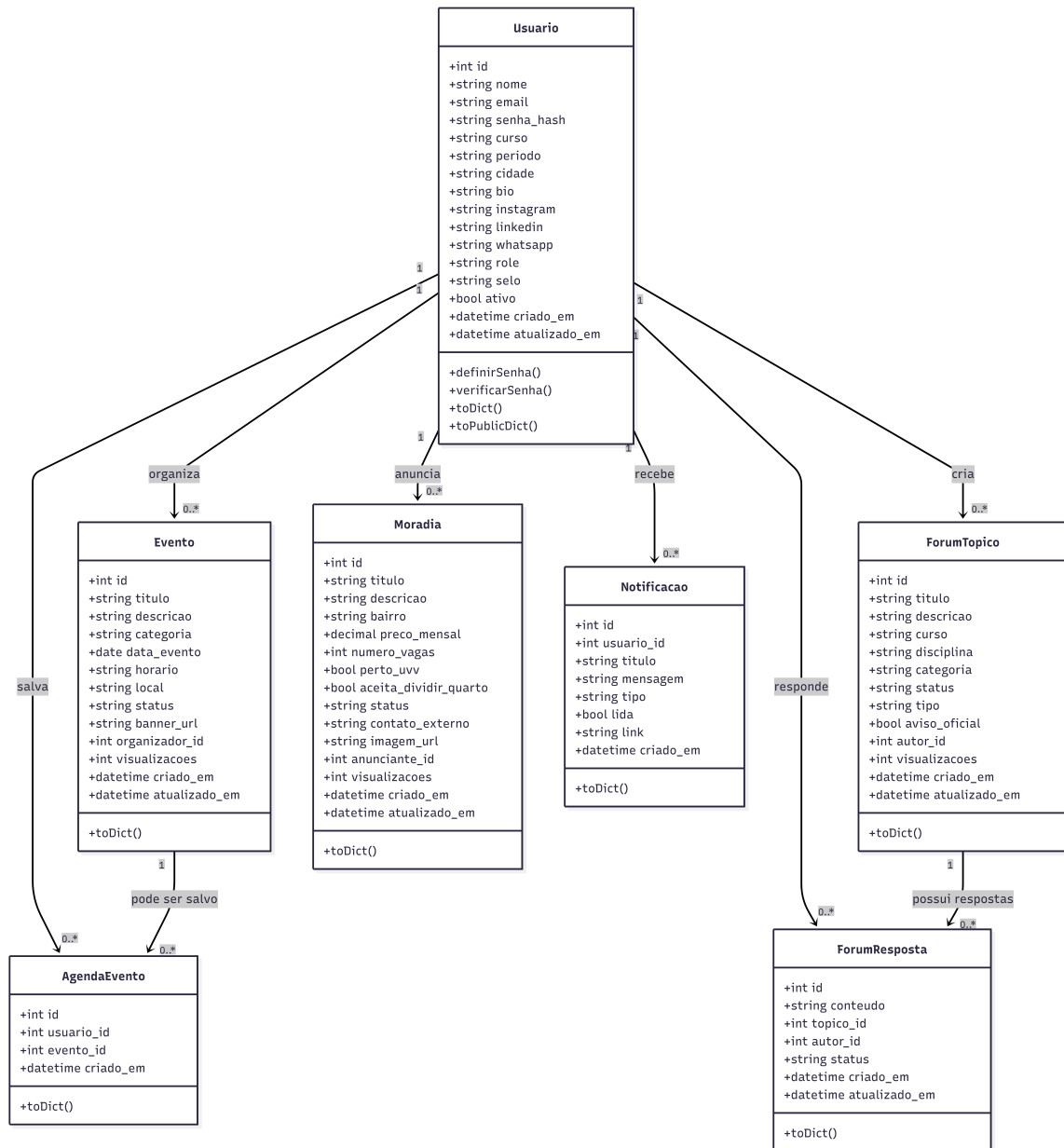


Figura 1: Diagrama do banco de dados do UniHub.

6.2 Tabelas Principais

- `usuarios`: dados de autenticação, perfil, role, selo e status.
- `cursos`: cursos cadastrados no sistema.
- `disciplinas`: disciplinas associadas aos cursos.
- `eventos`: eventos universitários e organizador.
- `evento_salvo`: relação entre usuário e evento salvo.
- `forum_topicos`: tópicos e avisos oficiais.
- `forum_respostas`: respostas dos tópicos.
- `moradias`: anúncios de moradia.
- `solicitacoes_moderador`: solicitações de permissão para moderação.
- `mensagens`: mensagens privadas entre usuários.
- `notificacoes`: notificações internas.

6.3 Regras de Integridade

- `usuarios.role`: `usuario`, `moderador` ou `admin`.
- `usuarios.selo`: `aluno`, `representante`, `atletica` ou `admin`.
- `eventos.status`: `ativo`, `cancelado` ou `encerrado`.
- `moradias.status`: `disponivel`, `pausada` ou `preenchida`.
- `forum_topicos.status`: `aberto`, `resolvido` ou `fechado`.
- `solicitacoes_moderador.status`: `pendente`, `aprovada` ou `recusada`.
- `evento_salvo`: impede que o mesmo usuário salve o mesmo evento mais de uma vez.

7 Testes Automatizados

Os testes usam `unittest`, SQLite em memória e seed automático. A classe base cria a aplicação em modo teste, sobe o banco, executa o seed e entrega um client Flask para os testes.

7.1 Cobertura Principal

- Autenticação e cadastro.
- Rotas públicas e protegidas.
- Eventos e agenda.
- Fórum e respostas.
- Moradias.
- Mensagens.
- Notificações.
- Usuários e roles.
- Regras de banco e constraints.

8 Tecnologias Utilizadas

- **Flask**: framework web principal.
- **Flask-Login**: sessão e autenticação.
- **Flask-SQLAlchemy**: ORM.
- **Flask-Migrate/Alembic**: versionamento do banco.
- **Flask-WTF/WTForms**: formulários e validação no servidor.
- **Jinja2**: templates HTML.
- **Bootstrap/CSS**: interface responsiva.
- **Flasgger**: documentação Swagger.
- **python-dotenv**: variáveis de ambiente.
- **unittest**: testes automatizados.
- **SQLite**: banco local e de testes.

9 Como Rodar o Projeto

9.1 Instalação Recomendada no Linux/macOS

```
python3 -m venv venv
source venv/bin/activate
python setup_project.py
```

9.2 Instalação Recomendada no Windows PowerShell

```
py -3 -m venv venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
python setup_project.py
```

9.3 Instalação Recomendada no Windows CMD

```
py -3 -m venv venv
venv\Scripts\activate.bat
python setup_project.py
```

O script deve ser executado com a **venv** ativa. Ele instala dependências, gera arquivos **.env**, aplica migrations e popula o banco inicial.

9.4 Instalação Manual no Linux/macOS

```
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
python make_env.py
flask --app app.py db upgrade
flask --app app.py seed
```

9.5 Instalação Manual no Windows PowerShell

```
py -3 -m venv venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
python make_env.py
flask --app app.py db upgrade
flask --app app.py seed
```

9.6 Rodar o Servidor

Linux/macOS ou Windows PowerShell:

```
flask --app app.py run --debug --port 5000
```

Com Invoke:

```
invoke setup
invoke dev
```

9.7 Executar os Testes

```
python -m unittest
```

9.8 URLs Principais

- Landing page: <http://127.0.0.1:5000/>
- Login: <http://127.0.0.1:5000/auth/login>
- Dashboard: <http://127.0.0.1:5000/dashboard>
- Swagger: <http://127.0.0.1:5000/swagger/>

9.9 Contas Fictícias para Teste

As contas abaixo são fictícias e foram criadas apenas para demonstração e testes do sistema em ambiente local.

- Administrador: `admin.unihub@example.com` / `senha123`
- Moderador: `moderador.unihub@example.com` / `senha123`
- Usuário comum: `usuario.unihub@example.com` / `senha123`

10 Principais Desafios e Soluções

- **Modularização:** o projeto foi dividido em `models`, `views`, `forms`, `services`, `utils` e `ext` para evitar rotas e regras misturadas em um único arquivo.
- **Controle de acesso:** roles foram padronizadas em `usuario`, `moderador` e `admin`, com helpers para proteger rotas sensíveis.
- **Validação:** formulários Flask-WTF foram adicionados para validar entradas no servidor e melhorar a confiabilidade da aplicação.
- **Persistência:** SQLAlchemy e migrations garantem modelagem explícita, constraints e reprodutibilidade do banco.
- **Testes:** a suíte usa banco em memória e seed automático para testar fluxos principais sem depender do banco local.
- **Entrega:** `.env.example`, README, migrations e seed foram mantidos para facilitar auditoria e execução pelo professor.